
MENU-DRIVEN STRING OPERATIONS USING C PROGRAMMING

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Aim

To write a menu-driven C program that performs string operations — converting a lowercase string to uppercase, reversing a string, and checking whether a string is a palindrome — using separate functions and without using any built-in string library functions.

2. Problem Statement

Write a menu-driven C program to perform the following operations using separate functions:

1. Convert a lowercase string to uppercase without using built-in functions.
2. Reverse a string.
3. Check if a string is a palindrome.
4. Exit.

3. Concepts Used

- **Strings in C:** a string is a one-dimensional array of characters that ends with the null character '\0'.
- **ASCII values:** every character has a numeric code. The lowercase letter 'a' has code 97 and the uppercase letter 'A' has code 65, so the difference between a lowercase letter and its uppercase form is always 32.
- **Functions:** each operation is written as an independent function, which makes the program modular, readable and easy to debug.
- **Menu-driven design:** a do-while loop keeps showing the menu and a switch statement selects the operation until the user chooses Exit.
- **Two-pointer technique:** one index starts at the beginning of the string and another at the end; they move towards each other. This is used for both reversing and palindrome checking.

4. Functions Used

The program is divided into the following functions. No function from `<string.h>` or `<ctype.h>` is used.

Function	Return Type	Purpose
<code>main()</code>	<code>int</code>	Shows the menu in a loop and calls the chosen function.
<code>displayMenu()</code>	<code>void</code>	Prints the list of available operations.
<code>readString()</code>	<code>void</code>	Reads a full line of text (spaces allowed) into an array.
<code>stringLength()</code>	<code>int</code>	Counts characters until <code>'\0'</code> (own version of <code>strlen</code>).
<code>toUpperCase()</code>	<code>void</code>	Converts lowercase letters of the string to uppercase.
<code>reverseString()</code>	<code>void</code>	Reverses the string in place by swapping characters.
<code>isPalindrome()</code>	<code>int</code>	Returns 1 if the string is a palindrome, otherwise 0.

5. Algorithms

5.1 Convert Lowercase to Uppercase

1. Start with index `i = 0`.
2. Repeat while `str[i]` is not the null character `'\0'`.
3. If `str[i]` lies between `'a'` and `'z'`, subtract 32 from it to get the uppercase letter.
4. Increase `i` by 1 and go back to step 2.
5. Stop. The string now contains uppercase letters (other characters are unchanged).

5.2 Reverse a String

1. Set `start = 0` and `end = length of string - 1`.
2. Repeat while `start < end`.
3. Swap `str[start]` and `str[end]` using a temporary variable.
4. Increase `start` by 1 and decrease `end` by 1.
5. Stop. The string is now reversed.

5.3 Check for Palindrome

1. Set `start = 0` and `end = length of string - 1`.
2. Repeat while `start < end`.
3. Convert the characters at `str[start]` and `str[end]` to lowercase (so that case is ignored).
4. If the two characters are different, return 0 (not a palindrome).
5. Increase `start` by 1 and decrease `end` by 1.
6. If the loop finishes without a mismatch, return 1 (palindrome).

5.4 Main Program (Menu)

1. Display the menu and read the user's choice.
2. If the input is not a number, show an error message and display the menu again.
3. For choices 1, 2 and 3, read a string and call the matching function, then print the result.
4. For choice 4, print a goodbye message and end the program.
5. For any other number, print "Invalid choice" and repeat from step 1.

6. Source Code

File name: string_operations.c

```
/*
 * Program : Menu-Driven String Operations
 * Author   : Rudra Pratap Singh
 * SAP ID   : 590032811
 * Batch    : 16
 *
 * Description:
 *   A menu-driven program that performs the following operations
 *   on a string, each in its own function and without using any
 *   built-in string function (no <string.h>, no toupper(), etc.):
 *   1. Convert a lowercase string to uppercase
 *   2. Reverse a string
 *   3. Check whether a string is a palindrome
 *   4. Exit
 */

#include <stdio.h>

#define MAX 100

/* Function prototypes */
int  stringLength(const char str[]);
void readString(char str[]);
void toUpperCase(char str[]);
void reverseString(char str[]);
int  isPalindrome(const char str[]);
void displayMenu(void);

int main(void)
{
    int  choice;
    char str[MAX];

    do
    {
        displayMenu();
        printf("Enter your choice (1-4): ");

        if (scanf("%d", &choice) != 1)
        {
            /* Invalid (non-numeric) input: clear the buffer */
            while (getchar() != '\n')
                ;
            printf("\nInvalid input! Please enter a number between 1 and 4.\n");
            continue;
        }
        while (getchar() != '\n') /* discard leftover newline */
            ;

        switch (choice)
        {
            case 1:
                readString(str);
                toUpperCase(str);
                printf("Uppercase string : %s\n", str);
                break;

            case 2:
                readString(str);
                reverseString(str);
                printf("Reversed string  : %s\n", str);
                break;

            case 3:
                readString(str);
                if (isPalindrome(str))
                    printf("\"%s\" is a PALINDROME.\n", str);
                break;

            case 4:
                return 0;
        }
    } while (choice != 4);
}
```

```

        else
            printf("\'%s\' is NOT a palindrome.\n", str);
            break;

        case 4:
            printf("\nExiting the program. Thank you!\n");
            break;

        default:
            printf("\nInvalid choice! Please enter a number between 1 and 4.\n");
    }
    while (choice != 4);

    return 0;
}

/* Displays the menu */
void displayMenu(void)
{
    printf("\n=====\\n");
    printf("        STRING OPERATIONS MENU\\n");
    printf("=====\\n");
    printf(" 1. Convert lowercase to UPPERCASE\\n");
    printf(" 2. Reverse a string\\n");
    printf(" 3. Check if a string is a palindrome\\n");
    printf(" 4. Exit\\n");
    printf("=====\\n");
}

/* Returns the length of a string (own implementation of strlen) */
int stringLength(const char str[])
{
    int i = 0;
    while (str[i] != '\\0')
        i++;
    return i;
}

/* Reads a line of text from the user (spaces allowed) */
void readString(char str[])
{
    int i = 0;
    int ch;

    printf("Enter a string: ");
    while ((ch = getchar()) != '\\n' && ch != EOF && i < MAX - 1)
    {
        str[i++] = (char)ch;
    }
    str[i] = '\\0';

    /* If the input was longer than MAX-1, discard the extra characters */
    if (ch != '\\n' && ch != EOF)
    {
        while ((ch = getchar()) != '\\n' && ch != EOF)
            ;
    }
}

/* Operation 1: converts lowercase letters to uppercase in place.
   In ASCII, 'a' - 'A' = 32, so subtracting 32 gives the capital letter. */
void toUpperCase(char str[])
{
    int i;
    for (i = 0; str[i] != '\\0'; i++)
    {
        if (str[i] >= 'a' && str[i] <= 'z')
            str[i] = str[i] - 32;
    }
}

/* Operation 2: reverses the string in place by swapping characters
   from both ends until the two positions meet in the middle. */

```

```

void reverseString(char str[])
{
    int start = 0;
    int end = stringLength(str) - 1;
    char temp;

    while (start < end)
    {
        temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end--;
    }
}

/* Operation 3: returns 1 if the string is a palindrome, otherwise 0.
   The comparison ignores case, so "Madam" is treated as a palindrome. */
int isPalindrome(const char str[])
{
    int start = 0;
    int end = stringLength(str) - 1;
    char a, b;

    while (start < end)
    {
        a = str[start];
        b = str[end];

        /* Convert both characters to lowercase before comparing */
        if (a >= 'A' && a <= 'Z') a = a + 32;
        if (b >= 'A' && b <= 'Z') b = b + 32;

        if (a != b)
            return 0;

        start++;
        end--;
    }
    return 1;
}

```

7. Sample Output

The program was compiled with gcc (using the -Wall -Wextra options, with no warnings) and tested with the inputs below. The number after "Enter your choice" and the text after "Enter a string" are typed by the user.

Output 1: Menu and Operation 1 – Convert to Uppercase

```

=====
          STRING OPERATIONS MENU
=====
1. Convert lowercase to UPPERCASE
2. Reverse a string
3. Check if a string is a palindrome
4. Exit
=====
Enter your choice (1-4): 1
Enter a string: hello world 123
Uppercase string : HELLO WORLD 123

```

Output 2: Operation 2 – Reverse a String

```
Enter your choice (1-4): 2
Enter a string: Programming
Reversed string : gnimmargorP
```

Output 3: Operation 3 – Palindrome Check

```
Enter your choice (1-4): 3
Enter a string: madam
"madam" is a PALINDROME.

Enter your choice (1-4): 3
Enter a string: hello
"hello" is NOT a palindrome.

Enter your choice (1-4): 3
Enter a string: Level
"Level" is a PALINDROME.
```

Output 4: Invalid Input and Exit

```
Enter your choice (1-4): 9

Invalid choice! Please enter a number between 1 and 4.

Enter your choice (1-4): abc

Invalid input! Please enter a number between 1 and 4.

Enter your choice (1-4): 4

Exiting the program. Thank you!
```

8. Test Cases

The following test cases were executed and all of them produced the expected result.

No.	Operation	Input	Output	Result
1	Uppercase	hello world 123	HELLO WORLD 123	Pass
2	Uppercase	Hello, C Programming!	HELLO, C PROGRAMMING!	Pass
3	Reverse	Programming	gnimmargorP	Pass
4	Reverse	a b c	c b a	Pass
5	Palindrome	madam	Palindrome	Pass
6	Palindrome	abba	Palindrome	Pass
7	Palindrome	Level	Palindrome	Pass
8	Palindrome	x	Palindrome	Pass
9	Palindrome	hello	Not a palindrome	Pass
10	Palindrome	Rudra	Not a palindrome	Pass
11	Menu	9	Invalid choice message	Pass
12	Menu	abc	Invalid input message	Pass

9. Conclusion

The menu-driven program was successfully developed and tested. Each operation — uppercase conversion, string reversal and palindrome checking — is implemented in a separate function without using any built-in string functions, which shows how string handling works at the character level. The project helped in understanding character arrays, ASCII values, loops, functions, the switch statement and the do-while loop. The program also handles invalid menu input, and the palindrome check ignores upper/lower case (so "Level" is accepted).